

Week 4-5 进度报告

固定模板压缩排序与单神经元准确率分析

2026-06-16

- <https://github.com/baipin/SpikeSort/blob/main/4.ipynb>
- <https://github.com/baipin/SpikeSort/blob/main/5.ipynb>
- <https://github.com/baipin/SpikeSort/tree/main/predictor>
- https://github.com/baipin/SpikeSort/tree/main/ratio_recommender

- 在阅读论文的过程中，发现了预处理同样包括CAR过程（Remove mean across time and median across channels），并且CAR应当在其他预处理操作之前进行，在之前的处理中遗漏了CAR操作。因此对之前的数据和操作都重新编写并运行了。

两周工作的整体逻辑

Week 4

用 baseline 数据上已经学好的模板，直接去匹配压缩-重建后的数据，评估压缩后排序效果。

Week 5

在固定压缩率下，把 Week 4 的总体评估下钻到 neuron 级别，得到每个神经元的敏感性与准确率；再进一步训练预测器与压缩率推荐器。

核心思想

不再重新执行 Kilosort 的漂移校正、模板学习、聚类与合并；而是**复用 baseline 模板与已有缓存**，只做 fixed-template matching 和计数统计。

输入与已有产物

- baseline Kilosort 输出: `templates.npy`, `ops.npy`, `spike_times.npy`, `spike_clusters.npy`
- 上游预处理结果: `whitened_data.npy`
- Week 4 缓存:
 - `week4_kilosort_function_fixed_template_results.npy`
 - `week4_ratio*.npy`
 - `week4_fixed_template_ratio_compare.csv`
- Week 5 缓存:
 - `week5_neuron_sensitivity_ratio*.npy/csv`
 - `predictor/week5_accuracy_predictor*`

Week 4: 要回答的问题

- ① baseline 上学到的模板，能否直接迁移到压缩重建数据？
- ② 如果不使用 baseline spike time 做候选锚定，只做严格的全时间轴匹配，效果如何？
- ③ 在不同压缩率下，排序性能如何变化？
- ④ 若 event-by-event 指标不稳定，是否能用更稳健的 binned spike-count accuracy 替代？

Week 4 Cell 0-4: 载入数据并构造固定模板匹配器

- Cell 0: 说明 notebook 主题是 “strict fixed-baseline-template sorting”。
- Cell 1: 导入 numpy/pandas/torch/matplotlib/pathlib 等依赖。
- Cell 2: 读取 baseline 模板与 spike labels; 把压缩重建 bin 映射成 (samples, channels)。
- Cell 3: 解释 383 vs 385 通道问题, 只保留 baseline 与重建数据的共享通道。
- Cell 4: 从 baseline ops 中取出 wPCA, 构造

$$U = \text{einsum}(\text{template}, \text{wPCA}),$$

并调用 `ks_tm.prepare_matching` 得到匹配所需张量 `ctc`。

Week 4 Cell 5–6: 严格全时间轴 fixed-template sorting

- Cell 5 首先尝试读取缓存 `week4_kilosort_function_fixed_template_results.npy`。
- 若缓存不可用，按窗口遍历整个压缩数据时间轴：
 - 每个窗口带 overlap，避免边界重复或漏检；
 - 对当前窗口调用 `ks_tm.run_matching`；
 - 只保留窗口中心区域检测结果；
 - 最后把全局时间排序。
- Cell 6 把结果保存回缓存，便于后续 notebook 直接复用。

说明

这里不是重跑完整 Kilosort，而是只使用 Kilosort 的 **template matching** 函数；不重新学习模板，不重新聚类。

Week 4 Cell 8–10: 严格排序结果的第一层评估

- Cell 8: 以 ± 12 samples 为容差，把压缩数据上的检测结果映射到最近 baseline spike，计算
 - 时间匹配覆盖率
 - 匹配后标签一致率
- Cell 9: 按 baseline cluster 汇总，导出每个 cluster 的 `n_ref`, `n_matched_from_test`, `same_label_rate`。
- Cell 10: 画出预测模板 ID 直方图、时间分布，并导出 CSV。

这一步的含义

它回答的是“压缩后数据里检测到的 spike，是否在时间上靠近 baseline spike”，但还不是最终 neuron accuracy 定义。

Week 4 Cell 11–13: 多压缩率与重建函数

- Cell 11: 提出多压缩率比较, 测试 0.10/0.20/0.30/0.50/0.70。
- Cell 12: 若某个重建 bin 缺失, 则从 `whitened_data.npy` 出发:
 - ① 对每个通道做 DCT;
 - ② 保留前 `keep_ratio` 比例系数;
 - ③ 做 IDCT 重建;
 - ④ 写回 `whitened_recon_ratio*.bin`。
- Cell 13: 封装三类关键函数:
 - baseline 时间点分类: `classify_at_baseline_times`
 - 全时间轴排序: `sort_full_timeline_with_fixed_templates`
 - binned accuracy 计算: `eval_binned_count_accuracy`

为什么不用 event-by-event 作为最终指标？

压缩后模板匹配容易出现时间抖动、局部过检或漏检。若只要求“同一时刻同一标签完全一致”，指标会过于苛刻，不能稳定反映神经元放电模式是否保留。

最终定义

把时间轴按固定 bin 分箱（本项目中为 1 秒），对每个 neuron、每个时间 bin 比较 baseline 与压缩结果中的 spike 数。

$$\text{matched}_{u,b} = \max \left(n_{u,b}^{\text{ref}} - \left| n_{u,b}^{\text{test}} - n_{u,b}^{\text{ref}} \right|, 0 \right)$$

$$\text{accuracy}_u = \frac{\sum_b \text{matched}_{u,b}}{\sum_b n_{u,b}^{\text{ref}}}$$

如何理解每个 neuron 的准确率

- 对于某个 neuron u , baseline 在第 b 个时间 bin 内有 $n_{u,b}^{\text{ref}}$ 个 spike。
- 压缩数据固定模板匹配后, 在同一 neuron ID、同一 bin 内得到 $n_{u,b}^{\text{test}}$ 个 spike。
- 两者差值越小, 说明该 neuron 的放电计数模式越被保留。
- 若某 bin baseline 为 50、压缩后为 55, 则该 bin 贡献

$$50 - |55 - 50| = 45$$

个 matched spikes, 该 bin 的局部准确率是 45/50。

解释

这个准确率衡量的是“该 neuron 的时间分箱发放强度是否保真”，而不是“每个 spike 是否逐一一一对应”。

不重跑 Kilosort 时，如何算出每个 neuron 的准确率

- ① 从 baseline Kilosort 结果读取 templates, spike_times, spike_clusters, ops。
- ② 用 baseline 模板和 wPCA 构造固定模板匹配器 U, ctc。
- ③ 对压缩重建 bin 只执行 ks_tm.run_matching, 得到压缩数据的

$$(st^{\text{test}}, clu^{\text{test}})$$

- ④ 不做新的漂移校正、模板学习、聚类合并，因此没有重跑完整 Kilosort pipeline。
- ⑤ 将 baseline 与压缩结果都按 neuron ID 和 1 秒时间 bin 计数。
- ⑥ 对每个 neuron 分别计算

$$\text{accuracy}_u = \frac{\sum_b \max(n_{u,b}^{\text{ref}} - |n_{u,b}^{\text{test}} - n_{u,b}^{\text{ref}}|, 0)}{\sum_b n_{u,b}^{\text{ref}}}$$

- ⑦ 把所有 neuron 的结果保存到 week5_neuron_sensitivity_ratio_*.npy, 以后直接读缓存即可。

不重跑 Kilosort 时，如何算出每个 neuron 的准确率

解释

即：

1. 固定 baseline 上已经得到的 neuron/template 身份
2. 看压缩重建后的数据，在这些固定身份下还能保留多少放电模式

所以这里没有“重新学模板、重新聚类”，而是在尝试回答这样的问题：“如果我拿 baseline 已经学好的模板，去压缩数据上做匹配，这些 neuron 的放电计数还像不像 baseline？”

Week 4 多压缩率总体结果

Ratio	Detection	Classif.	Binned Micro	Binned Macro
0.10	11.54%	23.83%	57.66%	54.56%
0.20	11.33%	23.57%	74.19%	66.00%
0.30	11.34%	23.33%	75.20%	66.97%
0.50	11.57%	23.31%	75.01%	66.76%
0.70	11.60%	23.39%	74.99%	66.74%

- event-level detection/classification 很低，说明固定模板直接迁移的 spike 级判别并不理想。
- 但 binned count accuracy 明显更高，说明对群体放电计数模式的保留比 event-level identity 更稳定。
- 在测试范围内，0.30 左右是比较平衡的压缩率，因此 Week 5 选 0.30 作为默认分析比率。

Week 5: 要回答的问题

- ① 在固定压缩率下，哪些 neuron 更敏感，哪些更稳健？
- ② 能否把总体 binned accuracy 细化为每个 neuron 的准确率与敏感性？
- ③ 能否进一步从 baseline 的 probe/neuron 特征预测未来压缩后的准确率？
- ④ 能否据此自动推荐一个合适的压缩率？

Week 5 Cell 0-4: 跨 notebook 复用已有产物

- Cell 0: 说明 Week 5 的主题是 fixed compression ratio 下的 neuron sensitivity。
- Cell 1: 导入依赖。
- Cell 2: 强调本 notebook 的定位是复用 earlier notebooks 的中间结果。
- Cell 3: 载入 baseline cache、compression cache、week4 strict cache, 以及 baseline Kilosort 原始输出。
- Cell 4: 枚举可用的 Week 4 ratio caches, 形成 cache overview。

结论

Week 5 的起点不是原始排序, 而是 Week 4 已经生成好的缓存与 baseline 结果。

Week 5 Cell 5–10: 单压缩率 neuron sensitivity

- Cell 5: 给出分析目标与指标定义。
- Cell 6: 默认选择 `analysis_ratio = 0.30`。
- Cell 7: 定义
 - `sort_full_timeline_with_fixed_templates`
 - `per_neuron_binned_accuracy`
- Cell 8: 与 Week 4 完全一致地构造固定模板匹配对象。
- Cell 9: 优先读取 `week5_neuron_sensitivity_ratio_0.30.npy`; 若缓存不存在才重新生成 neuron table。
- Cell 10: 只保留 `baseline_spikes > 0` 的 neuron, 按准确率排序。

Week 5: 每个 neuron 的表是怎么来的

- 每个 neuron 最终得到一行记录，字段包括：
 - neuron_id
 - baseline_spikes
 - compressed_spikes
 - matched_spikes_after_binning
 - binned_count_accuracy
 - sensitivity = 1 - accuracy
- 这些值全部来自 baseline spike train 与 fixed-template matching 输出的逐 bin 计数比较。

$$\text{sensitivity}_u = 1 - \text{accuracy}_u$$

解释

准确率越低，说明该 neuron 在压缩后越容易失真；敏感性越高。

Week 5 在 $\text{ratio}=0.30$ 的 neuron 级结果

- 活跃 neuron 数: 267
- mean binned neuron accuracy: 66.97%
- median binned neuron accuracy: 71.12%
- 最敏感 neuron: ID 246
 - baseline spikes = 106
 - compressed spikes = 548
 - matched spikes after binning = 0
 - accuracy = 0, sensitivity = 1
- 最稳健 neuron: ID 6
 - baseline spikes = 1079
 - compressed spikes = 1078
 - matched spikes after binning = 1078
 - accuracy = 99.91%

Week 5 Cell 11–13: 解释、可视化、导出

- Cell 11: 文字解释准确率与敏感性的含义。
- Cell 12: 三幅图
 - neuron accuracy 分布直方图
 - baseline spike count vs accuracy 散点图
 - 最敏感 15 个 neurons 柱状图
- Cell 13: 导出 `week5_neuron_sensitivity_ratio_0.30.csv`。

这里的实质输出

从这一刻开始，“每个 neuron 的准确率” 已经被缓存成结构化表格，可直接复用，无需再跑匹配。

Week 5 Cell 14–18: 从分析走向预测器

- Cell 14: 提出 accuracy predictor 的目标。
- Cell 15: 从 baseline 提取 neuron 特征，形成 predictor feature table。
- Cell 16: 对多个 ratio 组装训练集；若 `week5_neuron_sensitivity_ratio*.npy` 已存在则直接读取。
- Cell 17: 训练 ridge-style predictor，并做 leave-one-ratio-out 验证。
- Cell 18: 给出按文件接口调用 predictor 的示例。

要点

predictor 的监督信号，正是前面得到的 per-neuron binned_count_accuracy。

Week 5 Cell 19–22: 文档化接口与压缩率推荐

- Cell 19: 给出 `predictor/accuracy_predictor.py` 的调用说明。
- Cell 20: 说明压缩率推荐器 `ratio_recommender/best_ratio_recommender.py` 的新规则。
- Cell 21: 对单个 probe 输出推荐的 `keep_ratio`、对应预测准确率、是否满足门槛。
- Cell 22: 对多个 probe 批量输出一整组 `keep_ratio` 向量结果。

方法学闭环

Week 4 先定义并计算可解释的 neuron accuracy, Week 5 再把它变成可复用的预测与决策工具。

Week 5 新的压缩率推荐逻辑

旧逻辑

用

$$\text{predicted_accuracy} - \lambda \cdot \text{keep_ratio}$$

的惩罚得分做最大化。（已经弃用）

新逻辑

先设定一个最小可接受准确率阈值 `min_accuracy_threshold`，然后：

- ① 在给定区间内预测每个 `keep_ratio` 的 `probe` 平均准确率；
- ② 保留所有满足

$$\text{predicted_probe_accuracy_mean} \geq \text{threshold}$$

的 `ratio`；

- ③ 在这些可行解里选择最小的 `keep_ratio`；
- ④ 若整个区间都不满足门槛，则回退到预测准确率最高的 `ratio`。

新的推荐器输出形式

- 单 probe 输出字段为：
 - `recommended_keep_ratio`
 - `recommended_predicted_probe_accuracy_mean`
 - `threshold_feasible`
 - `selection_rule`
- 多 probe 批量接口直接输出

$$[r_1, r_2, \dots, r_n]$$

形式的 `keep_ratio` 向量结果。

- 在代码实现中，这个批量结果表对应 `keep_ratio_vector_table`。

解释

这个变化让推荐器从“折中打分”变成“先保证准确率底线，再尽量压缩”。

笔记本区段	已覆盖内容
4.ipynb 0–4	目标、依赖、baseline 读取、通道对齐、模板匹配器构建
4.ipynb 5–10	全时间轴 strict matching、缓存保存、时间邻近评估、cluster 汇总、可视化导出
4.ipynb 11–15	多 ratio 重建、函数封装、binned accuracy 定义、最终批量评估
5.ipynb 0–4	跨 notebook 数据导入、cache overview
5.ipynb 5–13	ratio 选择、per-neuron accuracy、缓存复用、排序、图表、CSV 导出
5.ipynb 14–22	predictor、训练集、模型训练、文件接口、threshold-based ratio recommender、keep_ratio 向量输出

本次汇报最关键的技术信息

- ① 单个 neuron 的准确率不是来自重新聚类，而是来自固定模板匹配后的时间分箱计数比较。
- ② Week 5 计算 neuron accuracy 时，优先直接读取 `week5_neuron_sensitivity_ratio*.npy`，因此通常连 matching 都不用重跑。
- ③ 如果缓存失效，最多只需重做 fixed-template matching；仍然不需要完整 Kilosort。
- ④ binned count accuracy 比 spike-by-spike 完全一致率更适合衡量压缩后神经元放电模式是否被保留。

- Week 4 建立了 baseline 模板迁移到压缩重建数据的评估框架。
- Week 5 把总体评估拆解为每个 neuron 的准确率与敏感性。
- 最核心的方法创新是：利用 baseline 模板 + 固定模板匹配 + binned counting，在不重跑完整 Kilosort 的情况下，得到每个 neuron 的准确率。
- 在现有结果中，ratio=0.30 兼顾压缩与保真，且 neuron 间差异显著，适合作为 predictor 与 threshold-based keep_ratio 推荐的训练支点。